

Reading a datasheet is for both the board and components, correct?

An embedded systems engineer should be able to read the datasheet of the entire board as well as the datasheets of the individual components used on it. This includes understanding the pin-level details of the microcontroller or processor and knowing how each pin functions and interfaces with other peripherals. Being comfortable with both board-level and component-level datasheets is essential for effective embedded software development.

Do engineering curricula teach students how to read and interpret datasheets, or is this a skill learned through specialised training programmes?

Many students claim experience in embedded systems because they have completed systems during their studies, but these are often not developed at the level the industry expects. They might know how to make something work using existing libraries or prebuilt code, but they usually lack a deeper understanding of how to read a datasheet, interpret it, and build software that interacts correctly with hardware at the pin and signal level.

So, to answer your question directly, this skill is generally not taught in college. Students typically begin learning it only when they join specialised training programmes or work in environments that emphasise real industry practices.

What skills do students need to become industry-ready?

Students become industry-ready by focusing on two essential areas: building skills and creating a strong portfolio. First, focus on skill development through practical assignments in areas such as bare metal programming, communication protocols, Linux internals, and embedded Linux application development. Second, build a solid portfolio. Skills matter only when backed by meaningful projects that show your ability to apply knowledge to real problems. Choose projects that reflect your understanding of microcontrollers, protocols, and how

hardware and software interact.

Domain-specific knowledge also plays a key role, for example, CAN or SPI in automotive applications and MQTT or COAP in IoT applications. Combining these skills and applying them through structured systems strengthens real-world problem-solving capability. By graduation, students should ideally have hands-on experience comparable to a professional. Strong skills, supported by a relevant systems portfolio, truly make students industry-ready.

Do recruiters today still focus heavily on marks, or do they tend to overlook grades if a candidate has strong technical skills?

Marks primarily serve as an entry filter for shortlisting curricula vitae. Once candidates reach the interview stage, problem-solving ability and technical depth take priority. With the growth of Global Capability Centres and Indian technology firms developing complete products, recruiters increasingly value hands-on capability and system contributions over grades.

A 7 to 8 CGPA is sufficient, but students should prioritise practical skills and real systems. Beginning early allows exploration of fields such as embedded systems or application software before specialising. Hackathons, internships, and open source participation help build credibility. A well-maintained GitHub profile often reflects consistency and skill more effectively than academic scores. Today, systems and problem-solving ability carry more weight than marks.

For software developers, GitHub is a great place to showcase work. Is it the same for embedded systems engineers?

The code you write is part of the project, but the main difference between embedded systems and web development lies in how results are showcased. GitHub is valuable for embedded engineers, though its presentation differs from that of web developers. While web developers can easily host and display live

systems on GitHub Pages, embedded engineers work with hardware that cannot always be displayed directly online. Instead, they can upload code, schematics, documentation, and demonstration videos. This approach still highlights coding skills, system organisation, and technical depth effectively.

How can students identify real-world challenges that could become opportunities for them to build solutions around?

Identifying real-world problems requires exposure to the right ecosystems. Students should participate in industry forums, idea-pitching events, and incubation programmes. Even attending as observers helps them understand market needs.

Students should also visit technology exhibitions, such as the annual Tech Summit in Bengaluru, where companies showcase innovations and industry trends. Exposure is the key. The more they interact with such ecosystems, the better they understand the problems and where their skills can have the most significant impact.

How can students use hackathons to make a strong portfolio?

Students should first identify what they aspire to become and then choose hackathons that align with those goals. Many hackathons focus on specific problems or themes, so selecting ones that align with their area of interest, such as embedded systems, adds real value.

That said, it is not only about winning. The key is participation and exposure. Hackathons allow students to test their ideas, learn from others, and understand real-world problem-solving. Even failures are valuable learning experiences. So, rather than being overly selective or competitive, students should view hackathons as part of their academic journey and as opportunities to build both skills and confidence over time. **EFY**

This interview is an excerpt of a longer discussion. For the full transcript, please visit: <https://efy.co.in/efyintfeb26>

